

HW4 API Paths — Customers, Orders, OrderDetails

This notebook calls the `/customers`, `/orders`, and `/orderdetails` routes from `app/main.py` over HTTP using the `requests` package.

Prerequisite: Run the API from the repository root, for example

`.venv\Scripts\python.exe -m app.main` (Windows) or `uvicorn app.main:app --reload --port 8000`. The default base URL is `http://127.0.0.1:8000`; override with the environment variable `API_BASE_URL` if you use another host or port.

Note: Cells that `POST`, `PUT`, or `DELETE` modify the underlying MySQL `classicmodels` database. The notebook cleans up after itself so it can be re-run.

```
In [1]: import os
import requests

BASE_URL = os.environ.get("API_BASE_URL", "http://127.0.0.1:8000").rstrip("/")

try:
    health = requests.get(f"{BASE_URL}/health", timeout=5)
    health.raise_for_status()
except requests.RequestException as exc:
    raise RuntimeError(
        f"Cannot reach API at {BASE_URL!r}. From the repo root run e.g. "
        f"`venv\Scripts\python.exe -m app.main`, then rerun this cell "
        f"(or set API_BASE_URL if the server uses another host/port)."
    ) from exc

# sample ids that exist in classicmodels
SAMPLE_CUSTOMER = 103
SAMPLE_ORDER = 10100
SAMPLE_PRODUCT_CODE = "S50_1392"
```

Customers

GET /customers

List with optional query parameters: `customerName`, `city`, `state`, `country`.

```
In [2]: resp = requests.get(f"{BASE_URL}/customers", timeout=30)
assert resp.status_code == 200, resp.text
data = resp.json()
assert "items" in data
print("count:", len(data["items"]))
data["items"][:3]
```

count: 122

```
Out[2]: [{'customerNumber': 103,
  'customerName': 'Atelier graphique',
  'contactLastName': 'Schmitt',
  'contactFirstName': 'Carine ',
  'phone': '40.32.2555',
  'addressLine1': '54, rue Royale',
  'addressLine2': None,
  'city': 'Nantes',
  'state': None,
  'postalCode': '44000',
  'country': 'France',
  'salesRepEmployeeNumber': 1370,
  'creditLimit': 21000.0},
 {'customerNumber': 112,
  'customerName': 'Signal Gift Stores',
  'contactLastName': 'King',
  'contactFirstName': 'Jean',
  'phone': '7025551838',
  'addressLine1': '8489 Strong St.',
  'addressLine2': None,
  'city': 'Las Vegas',
  'state': 'NV',
  'postalCode': '83030',
  'country': 'USA',
  'salesRepEmployeeNumber': 1166,
  'creditLimit': 71800.0},
 {'customerNumber': 114,
  'customerName': 'Australian Collectors, Co.',
  'contactLastName': 'Ferguson',
  'contactFirstName': 'Peter',
  'phone': '03 9520 4555',
  'addressLine1': '636 St Kilda Road',
  'addressLine2': 'Level 3',
  'city': 'Melbourne',
  'state': 'Victoria',
  'postalCode': '3004',
  'country': 'Australia',
  'salesRepEmployeeNumber': 1611,
  'creditLimit': 117300.0}]
```

```
In [3]: resp = requests.get(
    f"{BASE_URL}/customers", params={"country": "France"}, timeout=30
)
assert resp.status_code == 200
items = resp.json()["items"]
assert all(c["country"] == "France" for c in items)
print("french customers:", len(items))
items[:3]
```

french customers: 12

```

Out[3]: [{ 'customerNumber': 103,
  'customerName': 'Atelier graphique',
  'contactLastName': 'Schmitt',
  'contactFirstName': 'Carine ',
  'phone': '40.32.2555',
  'addressLine1': '54, rue Royale',
  'addressLine2': None,
  'city': 'Nantes',
  'state': None,
  'postalCode': '44000',
  'country': 'France',
  'salesRepEmployeeNumber': 1370,
  'creditLimit': 21000.0},
{ 'customerNumber': 119,
  'customerName': 'La Rochelle Gifts',
  'contactLastName': 'Labrune',
  'contactFirstName': 'Janine ',
  'phone': '40.67.8555',
  'addressLine1': '67, rue des Cinquante Otages',
  'addressLine2': None,
  'city': 'Nantes',
  'state': None,
  'postalCode': '44000',
  'country': 'France',
  'salesRepEmployeeNumber': 1370,
  'creditLimit': 118200.0},
{ 'customerNumber': 146,
  'customerName': 'Saveley & Henriot, Co.',
  'contactLastName': 'Saveley',
  'contactFirstName': 'Mary ',
  'phone': '78.32.5555',
  'addressLine1': '2, rue du Commerce',
  'addressLine2': None,
  'city': 'Lyon',
  'state': None,
  'postalCode': '69004',
  'country': 'France',
  'salesRepEmployeeNumber': 1337,
  'creditLimit': 123900.0}]

```

GET /customers/{customerNumber}

Returns **404** if the customer does not exist.

```

In [4]: resp = requests.get(f"{BASE_URL}/customers/{SAMPLE_CUSTOMER}", timeout=30)
assert resp.status_code == 200
resp.json()

```

```
Out[4]: {'customerNumber': 103,
        'customerName': 'Atelier graphique',
        'contactLastName': 'Schmitt',
        'contactFirstName': 'Carine ',
        'phone': '40.32.2555',
        'addressLine1': '54, rue Royale',
        'addressLine2': None,
        'city': 'Nantes',
        'state': None,
        'postalCode': '44000',
        'country': 'France',
        'salesRepEmployeeNumber': 1370,
        'creditLimit': 21000.0}
```

```
In [5]: missing = requests.get(f"{BASE_URL}/customers/99999999", timeout=30)
        assert missing.status_code == 404
        missing.json()
```

```
Out[5]: {'detail': "No customer with id '99999999'"}
```

POST /customers

Creates a customer. Response body is the new customer's primary key.

```
In [6]: new_customer = {
        "customerNumber": 90001,
        "customerName": "Notebook Test Co.",
        "contactLastName": "Tester",
        "contactFirstName": "Note",
        "phone": "555-0001",
        "addressLine1": "1 Test Street",
        "city": "New York",
        "country": "USA",
        "creditLimit": 1000.0
    }
    resp = requests.post(f"{BASE_URL}/customers", json=new_customer, timeout=30)
    assert resp.status_code == 200, resp.text
    resp.json()
```

```
Out[6]: '90001'
```

PUT /customers/{customerNumber}

Updates by primary key.

```
In [7]: updates = {"city": "Boston", "creditLimit": 5000.0}
        resp = requests.put(f"{BASE_URL}/customers/90001", json=updates, timeout=30)
        assert resp.status_code == 200
        print(resp.json())
        requests.get(f"{BASE_URL}/customers/90001", timeout=30).json()

{'updated': 1}
```

```
Out[7]: {'customerNumber': 90001,
        'customerName': '',
        'contactLastName': 'Tester',
        'contactFirstName': 'Note',
        'phone': '555-0001',
        'addressLine1': '1 Test Street',
        'addressLine2': None,
        'city': 'Boston',
        'state': None,
        'postalCode': None,
        'country': 'USA',
        'salesRepEmployeeNumber': None,
        'creditLimit': 5000.0}
```

DELETE /customers/{customerNumber}

Cleans up the test customer.

```
In [8]: resp = requests.delete(f"{BASE_URL}/customers/90001", timeout=30)
        assert resp.status_code == 200
        assert resp.json()["deleted"] == 1
        gone = requests.get(f"{BASE_URL}/customers/90001", timeout=30)
        assert gone.status_code == 404
        gone.json()
```

```
Out[8]: {'detail': "No customer with id '90001'"}
```

Orders

GET /orders

List orders with optional query parameters.

```
In [9]: resp = requests.get(f"{BASE_URL}/orders", timeout=30)
        assert resp.status_code == 200
        data = resp.json()
        print("count:", len(data["items"]))
        data["items"][:2]
```

count: 326

```
Out[9]: [{'orderNumber': 10100,
        'orderDate': '2003-01-06',
        'requiredDate': '2003-01-13',
        'shippedDate': '2003-01-10',
        'status': 'Shipped',
        'comments': None,
        'customerNumber': 363},
        {'orderNumber': 10101,
        'orderDate': '2003-01-09',
        'requiredDate': '2003-01-18',
        'shippedDate': '2003-01-11',
        'status': 'Shipped',
        'comments': 'Check on availability.',
        'customerNumber': 128}]
```

GET /orders/{orderNumber}

```
In [10]: resp = requests.get(f"{BASE_URL}/orders/{SAMPLE_ORDER}", timeout=30)
assert resp.status_code == 200
resp.json()
```

```
Out[10]: {'orderNumber': 10100,
        'orderDate': '2003-01-06',
        'requiredDate': '2003-01-13',
        'shippedDate': '2003-01-10',
        'status': 'Shipped',
        'comments': None,
        'customerNumber': 363}
```

```
In [11]: missing = requests.get(f"{BASE_URL}/orders/99999999", timeout=30)
assert missing.status_code == 404
missing.json()
```

```
Out[11]: {'detail': "No order with id '99999999'"} 
```

OrderDetails

OrderDetails uses a **composite primary key** (orderNumber + productCode), so the path needs both pieces.

GET /orderdetails

```
In [12]: resp = requests.get(f"{BASE_URL}/orderdetails", timeout=30)
assert resp.status_code == 200
data = resp.json()
print("count:", len(data["items"]))
data["items"][:2]
```

count: 2996

```
Out[12]: [{'orderNumber': 10100,  
          'productCode': 'S18_1749',  
          'quantityOrdered': 30,  
          'priceEach': 136.0,  
          'orderLineNumber': 3},  
          {'orderNumber': 10100,  
          'productCode': 'S18_2248',  
          'quantityOrdered': 50,  
          'priceEach': 55.09,  
          'orderLineNumber': 2}]
```